

KeyQuorum Lab

Browser GUI Manual

Bailey Forbes

A guide to KeyQuorum Lab's own browser interface: its header and tabs, and the guided, gated tutorials built into the page. For the `keyquorum` / `keyquorum-device` command-line tool the lab runs underneath, see the separate KeyQuorum manual.

Contents

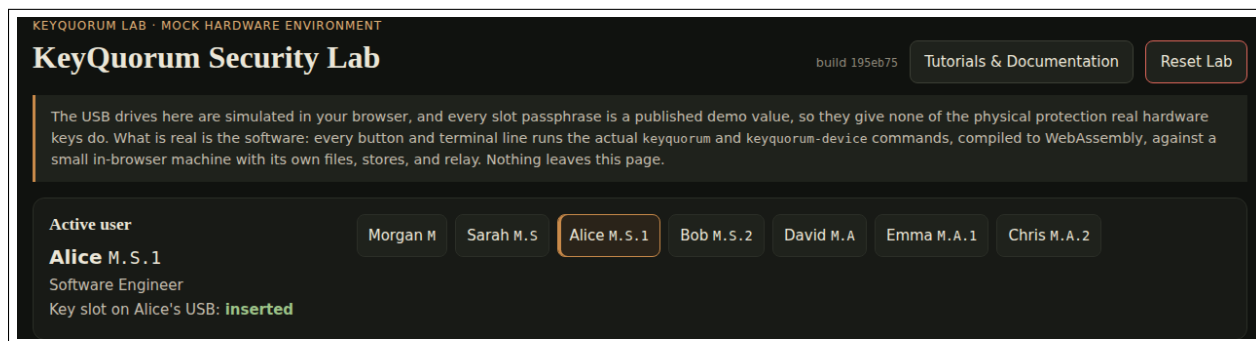
1	Overview	1
1.1	Default credentials	1
1.2	Identity and drives	2
2	The tabs	2
2.1	Organization key tree	2
2.2	Mock USB devices	3
2.3	File Explorer	4
2.4	Inbox and sent	4
2.5	Security & devices	5
2.6	Activity and access trace	6
2.7	Terminal	8
3	Guided tutorials	9
3.1	How a step works	10
3.2	The modules	11
3.2.1	Identities & drives	11
3.2.2	Files & unlocking	11
3.2.3	Mailbox: sending & receiving	11
3.2.4	File history	11
4	Where the lab lives	12

1 Overview

KeyQuorum Lab runs the KeyQuorum crate in a browser as a small sandboxed machine, compiled to WebAssembly. It has its own files, a SQLite store per person, mock USB drives, and a mailbox relay answering in the page. Every button and every terminal line runs the real `keyquorum` and `keyquorum-device` commands — there is no separate “demo mode” standing in for them.

Note

This manual documents the lab’s own GUI: its header, tabs, and guided tutorials. It does not re-document `keyquorum` / `keyquorum-device` themselves — that is the separate *KeyQuorum* manual, which the lab runs underneath its interface unchanged. Where a control here is a thin wrapper over a CLI concept (custody, quorum, private bridges, authenticated updates, the mailbox relay), this manual names that concept without re-explaining it.



The lab header: build commit, the Tutorials & Documentation and Reset Lab buttons, the disclaimer, and the Active user bar.

Reset Lab discards everything the session has done — every drive move, sent letter, provisioned key, password file, export, share link, signature, and activity-log entry — and replaces it with a fresh copy of the seeded demo state, instantly. Nothing here is persisted outside the browser tab, so this is the only way back to a known starting point; it is also what **Exit tutorial** and the tutorial picker leave alone; a gated tutorial in progress does not survive a reset.

Tasks that take several commands in a plain shell take one here — a file is sent with `keyquorum send`, a letter is opened and answered with `keyquorum inbox open`, and `keyquorum doctor` says what is missing — and the lab shows each command it ran. The older, longer procedures (`deliver`, `file share|receive|ack`, `relay pull`) still work if typed in the Terminal tab, but no button uses them, and they are not recommended (Section 2.7).

The lab itself is a browser-based virtual machine: its mock USB drives emulate USB-token custody entirely in memory, and every slot passphrase is a published default value (Section 1.1), so they give none of the physical protection real hardware keys do. What is real is the software underneath: every action, in every tab, is one of the same commands documented in the KeyQuorum manual.

1.1 Default credentials

Every credential the lab uses is a published placeholder, not a secret. The lab ships as a public WebAssembly bundle that runs entirely in your browser, so anything baked into it — the seeded people, their USB passphrases, and the seeded files — is visible to anyone who inspects the bundle. That is by design: none of it guards real data, so nothing here needs to stay confidential, and no real file or hardware key is ever at risk of passing through the lab.

USB slot passphrases. Every seeded USB slot unlocks with `lab-demo-<label>`, where `<label>` is that slot’s dotted tree label:

Person	Label	Passphrase
Morgan	M	lab-demo-M
Sarah	M.S	lab-demo-M.S
Alice	M.S.1	lab-demo-M.S.1
Bob	M.S.2	lab-demo-M.S.2
David	M.A	lab-demo-M.A
Emma	M.A.1	lab-demo-M.A.1
Chris	M.A.2	lab-demo-M.A.2

Terminal default values. Typing a command yourself at the Terminal tab — rather than clicking a GUI action that stages your own value first — is answered with the same published defaults a real prompt would use, whenever it asks for a password or PIN:

- Password: `lab-demo-password`
- PIN: `0000`

Note

Seeded quorum-protected files never use a password. They are protected by KeyQuorum’s quorum and device-custody policies instead, and are unlocked with the USB identities above.

1.2 Identity and drives

The **Active user** bar shows who you are acting as right now, and which mock USB drive their personal key slot lives on. Clicking another name’s chip switches to them, the way sitting down at a different desk would. Most actions need that person’s drive *inserted* first — a mock drive behaves like a real one: its slots are only usable while it is connected, and moving a slot onto someone else’s drive needs both drives inserted at once, matching the physical requirement of moving a token between two real USB drives.

2 The tabs

At desktop width every tab’s panel is visible at once, stacked on one scrolling page; the tab row itself only appears at phone width, where it switches which single panel is shown. What follows is every panel, in the order they appear on the page.

2.1 Organization key tree

The org-wide split tree, drawn as the dotted-label hierarchy it is: `M`, `M.S`, `M.S.1`, and so on. You only ever see your own slice — your lineage, your descendants, siblings, and established-bridge peers. A leaf tagged *required* contributed a share the last time you unlocked a file; *satisfied* means that share was actually presented.

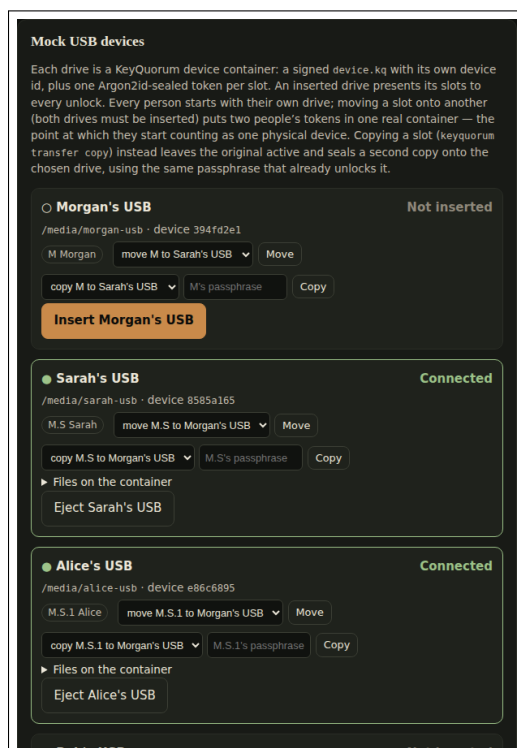
- **Revoke key** bans a leaf’s hardware key outright; it drops that leaf’s bindings and pairings but does not evict it or refresh survivor shares.
- **Reissue...** replaces a leaf’s hardware key onto an already-provisioned replacement token.
- **Bridges** whitelists and establishes cross-branch links, the same two-step allow-then-add process the CLI uses.
- **Tree restructure** walks through proposing a new public generation and collecting the parent’s countersignature when one is required.



The organization key tree, showing the dotted-label hierarchy, per-leaf slot status, and the Revoke/Reissue controls.

2.2 Mock USB devices

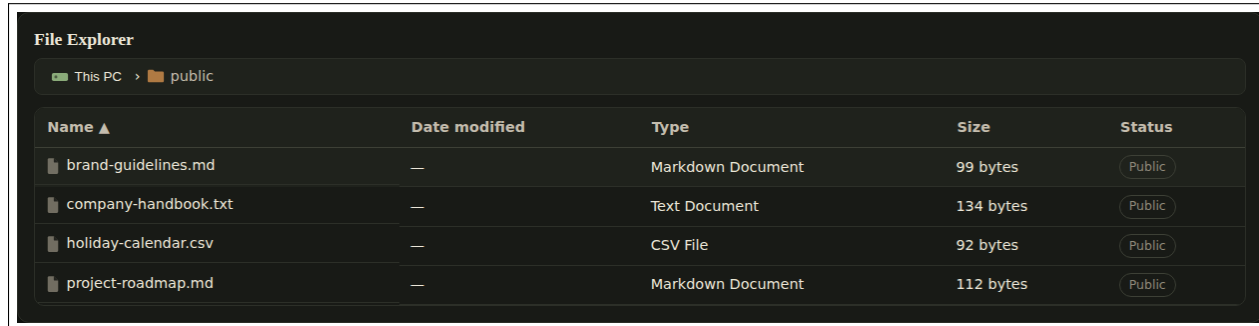
Each card is one device container: a signed device descriptor plus one sealed token per slot. **Insert** / **Eject** plug a drive in or pull it out. **Move** asks for a destination drive (both drives must be inserted) and relocates a slot's keypair onto it — the point where two people's tokens land in one physical container and start counting as a single device toward any multi-device quorum. **Copy** instead leaves the original slot active and seals a second copy elsewhere, using the same passphrase that already unlocks it — a seeded slot's own published default passphrase (Section 1.1).



Mock USB devices: connected state, the slots each drive carries, and the Move / Copy controls.

2.3 File Explorer

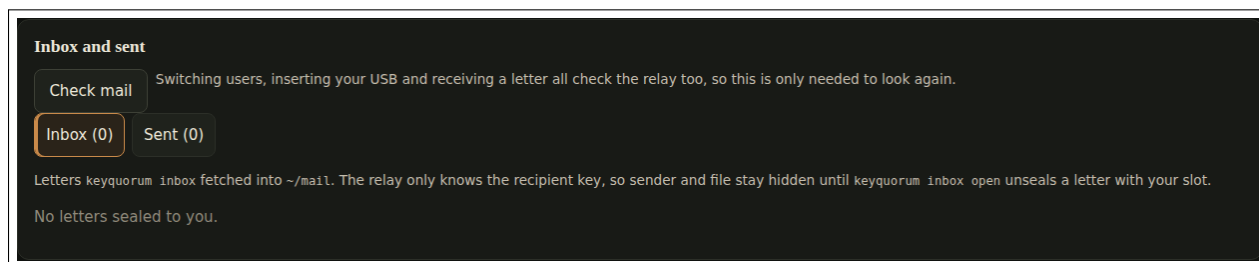
A Windows-Explorer-style view onto the active user's files, grouped into folders. Double-clicking a file (or selecting it and pressing **Open**) unlocks and views it — a quorum file prompts for the shares it needs, a password file for its password. **Send...** seals a copy to another label's key and drops it in their inbox; **Sign** is only offered on public or received plaintext; **Properties** shows the file's protection, size, and timestamps. Right-clicking a row offers the same actions. A file with an expiry stamp shows a status pill that flips to *expired* on its own while the tab is open.



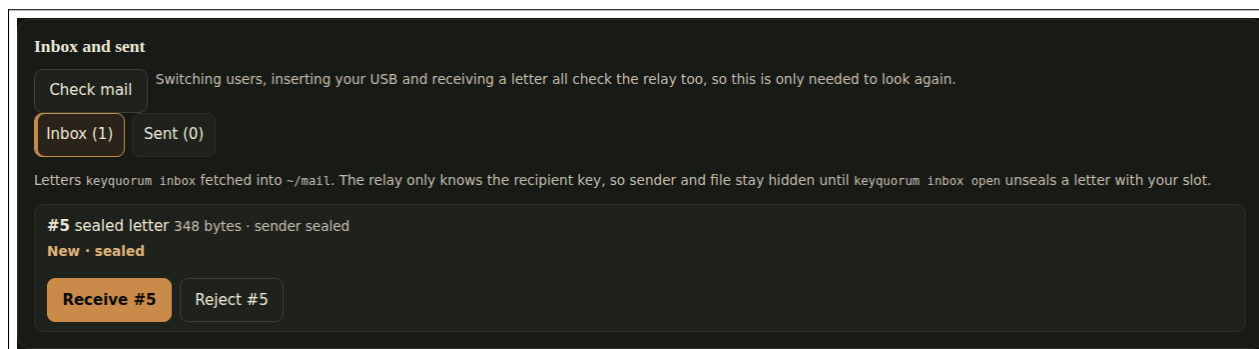
The File Explorer, showing the public folder's contents and their status pills.

2.4 Inbox and sent

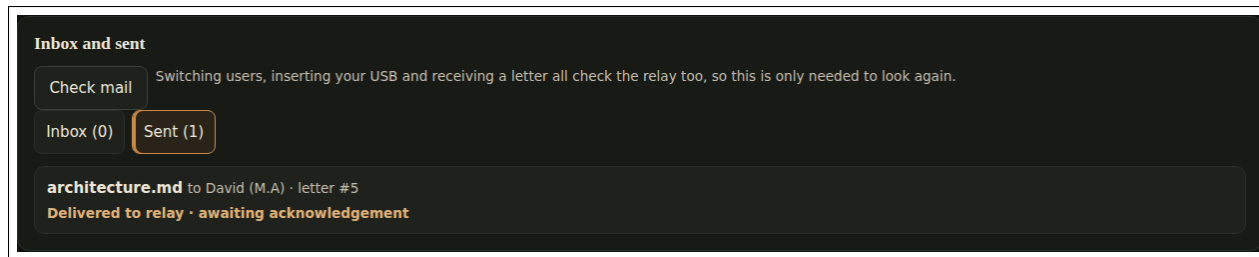
Letters are sealed to a recipient's key and routed by the mailbox relay without it ever reading them — sender and contents stay hidden until **Receive** unseals a letter with your own slot. **Reject** answers with a signed refusal instead; **Receive** checks the relay first (`keyquorum inbox list`, then `inbox open`), so there is nothing to refresh. The Sent view tracks whether a delivery is still awaiting the recipient's signed acknowledgement. Acknowledgements are opened by themselves whenever the sender is signed in with their USB inserted — on switching user, on inserting the drive, and after receiving — and **Check mail** looks again on demand. If the sender's USB is out, the answer waits, sealed to their key.



Inbox and sent, in the empty seeded state, with the **Check mail** button.



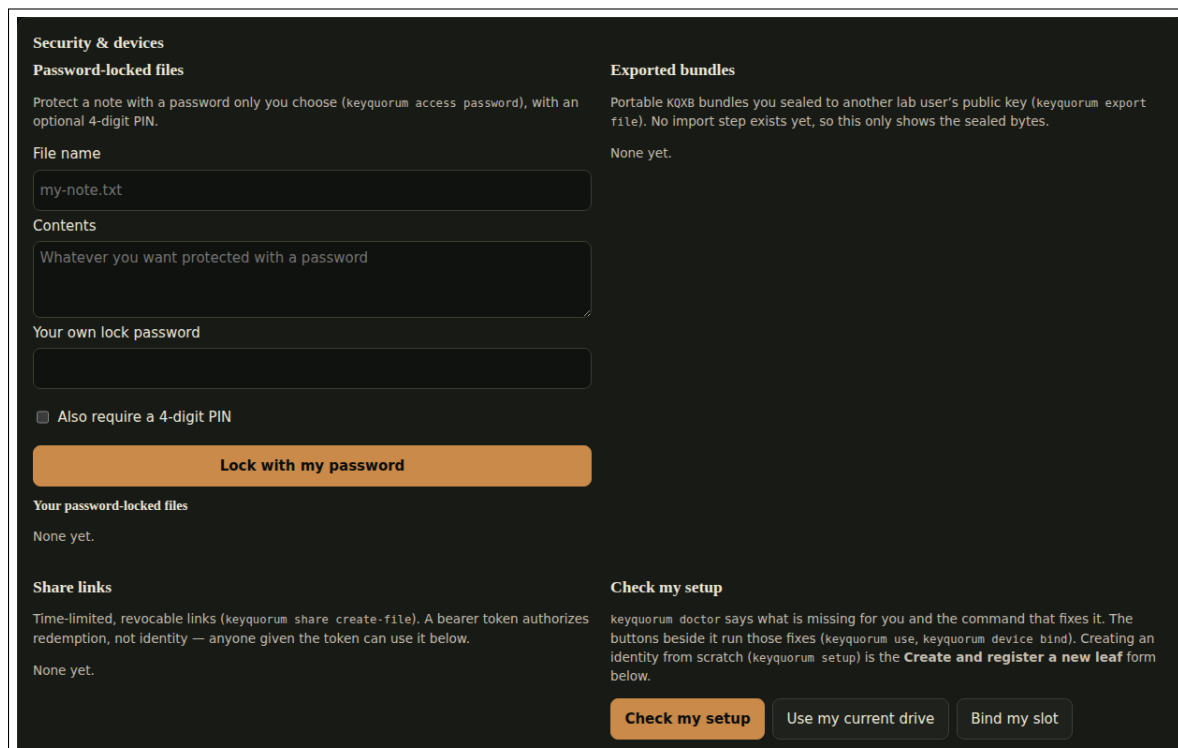
David's inbox with a sealed letter waiting: sender and file stay hidden until **Receive** opens it with his slot.



Alice's Sent view after David received the letter: the delivery is marked acknowledged.

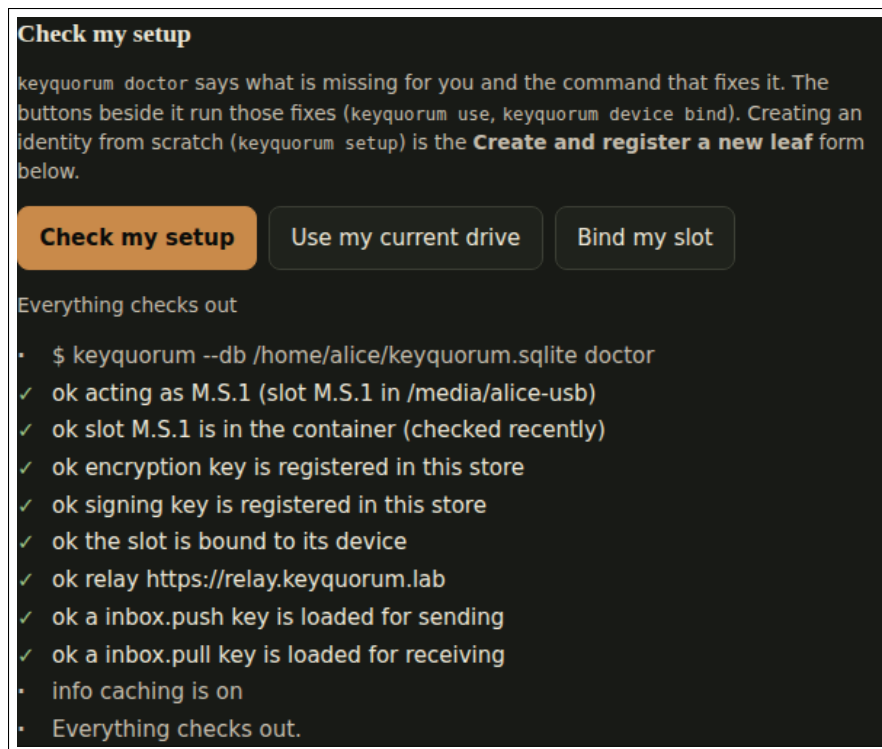
2.5 Security & devices

Everything that is not the hardware-key quorum tree or a file: locking a note with a password (and optionally a PIN), exporting a file or credential as a portable sealed bundle, minting a time-limited share link, **Check my setup** (`keyquorum doctor` for you, with **Use my current drive** and **Bind my slot** for the fixes it names), creating a key on an inserted drive and registering it as a new leaf in one step (**Create and register a new leaf**; the separate provision and register forms stay under *Separate steps*), signing or verifying with a private sign bridge you belong to, and, further down, read-only device logs and relay counters for the in-process mailbox this lab runs against.

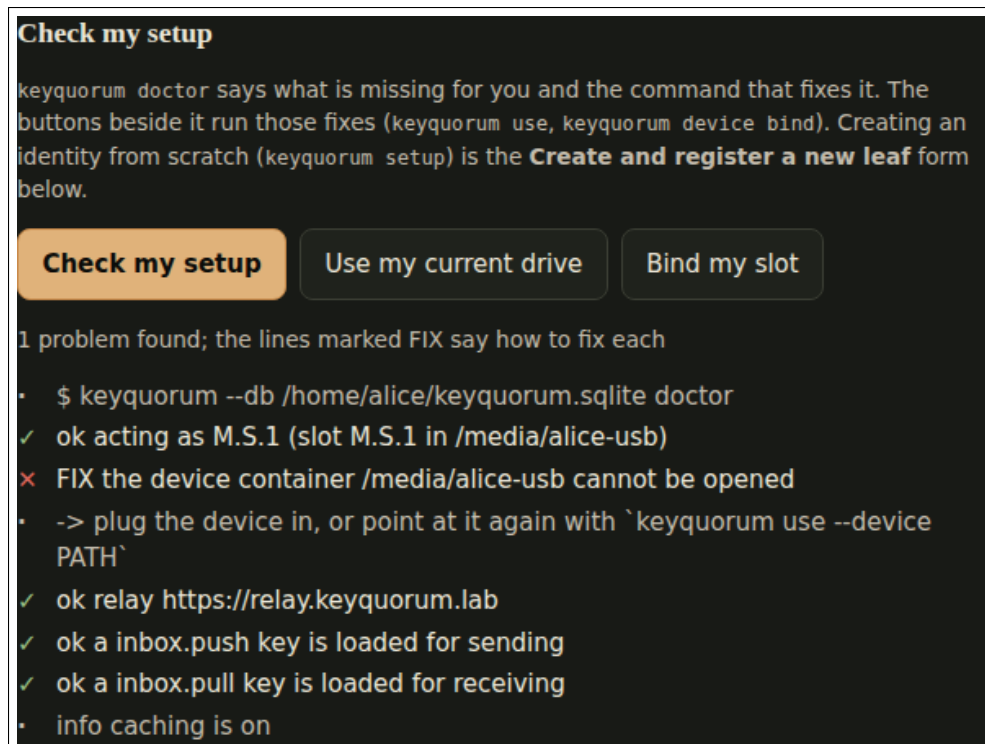


Security & devices: the password-lock form, exported bundles, share links, and the **Check my setup** block.

Check my setup prints what `keyquorum doctor` found for the active person: a line for each thing that works, and for each thing that does not, a *FIX* line with the command that repairs it. **Use my current drive** (`keyquorum use`) and **Bind my slot** (`keyquorum device bind`) run those fixes. Everyone seeded already has both done; the lab re-runs them for the owner when a slot is moved to another drive. `keyquorum setup` — the command that does all of it from nothing — is not a button here, because the lab's people are fixed. The **Create and register a new leaf** form below is a different, combined workflow: it provisions a slot on an inserted drive, registers both keys, binds the slot and adds the leaf to the organization tree.



Check my setup with everything in place: every line passes.



The same check with Alice's USB out: a FIX line says the container cannot be opened and how to repair it.

2.6 Activity and access trace

Every action taken leaves a trace here: which checks ran, in order, and whether each one passed. The **Equivalent operation** line under the latest trace is the literal command line that action ran — the same one that could be typed in the Terminal tab. The full log below keeps every action for the session, not just the last one.

Tracked files. The Activity tab is also where tracked files live. The **Tracked files** panel acts as whoever is active, with their own store and their own drive's slot; each button runs one real **keyquorum file** command:

- **Track and sign** a new file, then **Check in** edits — signed with your slot, or unsigned to see a revision stay *pending*.
- Per revision: **Sign revision** (its author), **Approve as parent** (the author's direct parent countersigning), **Finalize** (the scope owner or an ancestor, once it is trusted), **View** an older revision, and **Diff** it against its parent. Each revision shows its user label first, when it has one, then its generated label and short id.
- **Rename file**: change the name shown; the file id and every revision id stay the same.
- **Share file** with another person. The line above the button says which revision would go — the head, or the last trusted revision when the head is pending. The letter, and every answer, travels through the relay (**send**, then **inbox open**); no shared folder is involved. The recipient **Accepts** or **Refuses** the letter (**inbox open ID -out**, or **-reject**); the sender's copy records the answer by itself (**inbox open ID -file**) when the sender is next signed in with their USB inserted. **Record the answer** appears only while it is still waiting.
- **Ask for a file, or for a change**: pick a person, choose *a file* (send me your copy) or *a change to this file*, add a message if you like, and press **Send request** (**keyquorum file request**, with **-change** for a change). Under *Requests for a file or a change* the holder sees the message and presses **Accept request** or **Decline request** (**inbox open ID -accept** or **-decline**, with **-file** naming their own copy so the request is recorded there). The asker's copy records the answer by itself (**inbox open ID -file**) when the asker is next signed in with their USB inserted. A request only asks: nothing is delivered or changed. To serve an accepted file request the holder uses **Share file**; a change arrives as an ordinary revision. The log shows the request id, kind and decision, never the message.
- When two copies diverge: **Import copy**, **Review the fork** (each side's changed lines and the named reviewer) and **Merge heads**; a clean merge is a new revision that still has to be signed. **Resolve** runs **keyquorum file resolve** as the active person with their own slot: keep the left or right side, use text you type, or reject a proposed merge. Only the assigned reviewer (or an ancestor of theirs) can settle a conflict, so the same button is refused for anyone else; the lab adds no check of its own.
- **Verify history**, **Export history snapshot** and **Check** a snapshot against the file.
- **Schedule expiry** or **Destroy content now**: the file becomes a tombstone that still verifies.
- **Link gate**: record unlocks of a quorum file, or of one of your password files, in this file's history; **Unlink** stops it.

Whatever a button or a Terminal command writes to a tracked file appears in the log at once, right below the action that caused it. History entries can be expanded (revision, labels, parents, trust, history root) and filtered by **Tracked File**, **Revision**, **Security**, **Sharing** and **Conflict**; a revision graph per file sits under the log. Three tracked files are seeded: **budget.txt** (an unsigned edit, so sharing falls back), **forecast.txt** (two edits that merged on their own) and **memo.txt** (two edits to one line, assigned to a reviewer).

Tracked files
Acting as Sarah (M.S). Each button runs a real keyquorum file command with your slot, what it records appears in the log below.

New tracked file

index.txt
First revision's text

Track and sign

Tracked file: budget.txt — /srv/keyquorum/tracked/budget.txt.kqtf

budget.txt scope M.S · 6 events · root 83488187 · /srv/keyquorum/tracked/budget.txt.kqtf

Q4 baseline Rbudget-20261002T223856.336Z-M.S d5180a3d by M.S (root) TRUSTED View Finalize

Late edit, unsigned Rbudget-20261002T223856.343Z-M.S 5bbdea48 by M.S ← d5180a3d · head PENDING MissingContentSignature View Diff Sign revision

Current revision 5bbdea48; latest trusted d5180a3d; nothing final yet.
The head is not trusted, so sharing sends d5180a3d, the last trusted revision.

Verify history
Export history snapshot

Record unlocks of (architecture.md (quorum)) Link gate

Edit the current revision

Q4 budget: 125000

Short description (optional)

☒ Sign this edit with my slot

Check in

New name Rename file

Expires (UTC) mm/dd/yyyy, --:--:-- Schedule expiry Destroy content now

Ask Morgan (M)

for a file (send me your copy)

Message (shown to them, never recorded)

Send request

Share with Morgan (M) Share file

The Activity tab's tracked files panel: revisions with their trust, what sharing would send, and the controls for the selected file.

A hand-over, as the panel shows it. Sarah shares `budget.txt` with Alice and the letter waits; Alice accepts it into her own copy; when Sarah is next signed in with her USB, her copy records Alice's answer without a click.

Tracked-file letters

budget.txt: Sarah → Alice WAITING

Sarah's side: the letter to Alice is waiting.

Tracked-file letters

budget.txt: Sarah → Alice WAITING Accept budget.txt Refuse budget.txt

Alice's side: **Accept** or **Refuse** the letter.

Tracked-file letters

budget.txt: Sarah → Alice ACCEPTED · answer recorded

Back as Sarah: the answer is recorded.

Requests for a file or a change

budget.txt: Sarah asks Alice for a change WAITING · "Please tighten the second paragraph" Accept request for budget.txt Decline request for budget.txt

A request for a change, as the holder sees it: the message is shown to her and never recorded.

2.7 Terminal

A real shell on the same lab machine. Every button on every other tab runs a command here under the hood; typing it directly works identically, including commands no button exposes yet. Use full paths in `keyquorum` arguments: the lab's shell does not expand `~` inside them, so `-db ~/keyquorum.sqlite` would open a new, empty store rather than your own.

```
keyquorum-device list /media/alice-usb
bridge list 1
usb insert david
```

```
su david
```

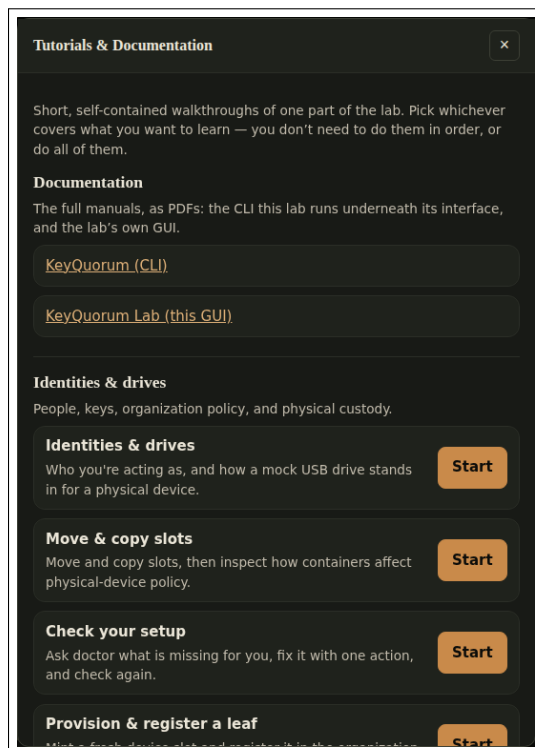
`usb insert` / `usb eject` and `su` are lab-only shell verbs for drive and identity control; every other line is an ordinary `keyquorum` / `keyquorum-device` command.

A few shorthands save typing the database path: `use`, `setup` and `doctor` run as the active person against their own store, `bridge` runs against the organization store, `send FILE USER` sends a file, `inbox` lists the letters waiting, and `inbox open [ID]` opens one letter, or every new one, and posts the answers. `receive ID` is `inbox open ID` and `reject ID` is `inbox open ID -reject`.

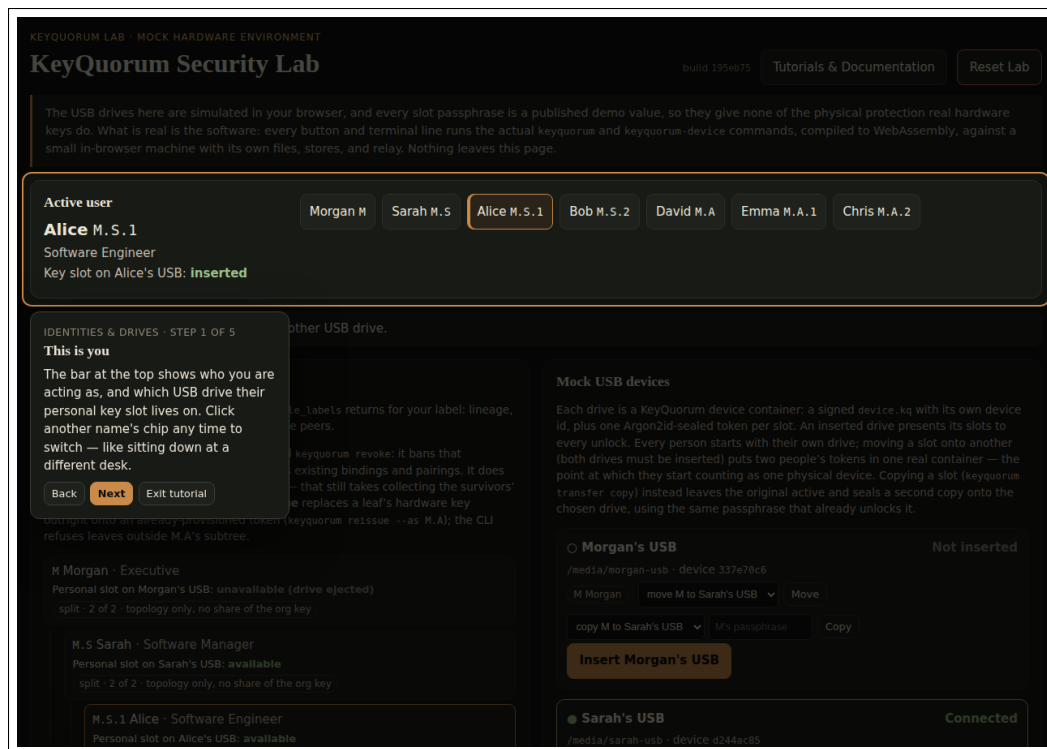
Legacy commands. The older commands — `deliver send`, `deliver open`, `deliver ack`, `file share`, `file receive`, `file ack`, `relay pull` — still work if you type them here, exactly as in the CLI manual, and they print the same note naming their replacement. They are not recommended, and no button in the lab uses them: a full `keyquorum ... inbox open` line typed by hand also bypasses the lab's own mailbox bookkeeping, which the shorthand does not. Any passphrase, password, or PIN prompt a typed command raises is answered with its published default value (Section 1.1), the same way a GUI action's prompt is when you have not staged your own answer first.

3 Guided tutorials

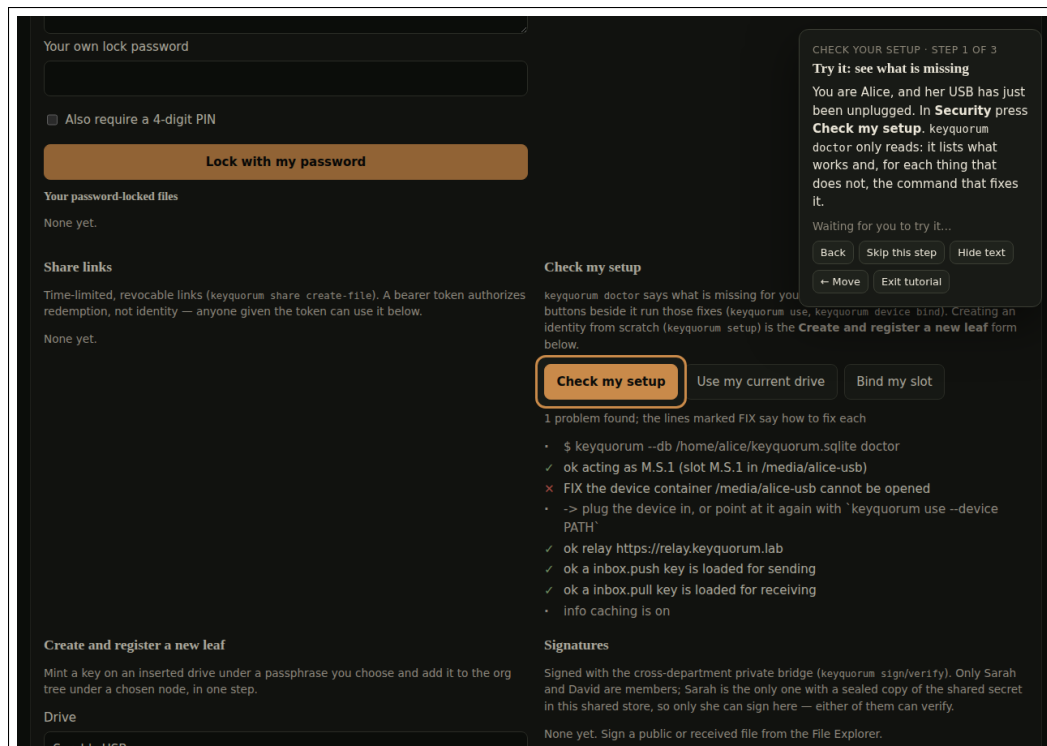
The **Tutorials & Documentation** button in the header opens a picker of short, self-contained modules, plus a **Documentation** section linking the same two PDF manuals this crate ships everywhere else — the CLI manual and this one. Each module covers one part of the lab; picking one is not a commitment to the others, and they can be taken in any order.



The picker: its Documentation section (both PDF manuals) above the Identities & drives category.



A tutorial step in progress: the Active user bar is spotlighted, and the tooltip names the step and what to do.



The Check your setup module: Alice's USB is out, so the tooltip asks for **Check my setup** and the result lists the FIX line.

3.1 How a step works

Each step spotlights the part of the page it is about and shows a tooltip with an explanation. A step comes in one of two kinds:

- An **informational** step just explains something; a **Next** button advances it.

- A **gated** step (marked “Waiting for you to try it...”) names a real action to perform — insert a drive, open a file, switch identity, send a letter — and does not advance until that action actually happens in the lab. Completion is checked against the lab’s own state (the current snapshot, or the most recent entry in the Activity log), not against a click on the tooltip itself, so there is no way to fake having done it. Once satisfied, the step marks itself done and advances on its own a moment later. **Skip this step** is always available if you would rather move on without doing it.

A step can also relocate its target as the page changes under it — for example, tracking a folder tile in the File Explorer until it is opened, then tracking the file row that takes its place — so one step can walk you through a short sequence rather than only ever pointing at one fixed element.

3.2 The modules

The picker groups its nineteen modules into four categories; taking one module is never a commitment to the others in its category, or to the category itself.

3.2.1 Identities & drives

- **Identities & drives** — who you’re acting as, bringing a mock USB drive online, reading the organization key tree’s own slice, and switching identity.
- **Move & copy slots** — move and copy slots, then inspect how containers affect physical-device policy.
- **Check your setup** — run `keyquorum doctor` while Alice’s USB is out, plug it back in, and run it again to see it go green.
- **Provision & register a leaf** — create a fresh device slot and register it in the organization tree in one step.
- **Key administration** — revoke, reissue, restructure, countersign, and understand parent approval.
- **Bridges & remote devices** — manage cross-branch visibility and distinguish local device actions from relay transfer.

3.2.2 Files & unlocking

- **Files & unlocking** — opening a public file, then trying a quorum-protected one to see what the access trace looks like when a single identity does not meet its threshold.
- **Complete a quorum unlock** — read a file’s requirements, connect cooperating devices, and complete an unlock.
- **Passwords & PINs** — lock a note with your own password and an optional PIN, unlock it again to see the lab’s second, independent protection scheme end to end, then lock a second file straight from the Terminal and watch a typed command fall back to the published default password and PIN (Section 1.1) instead of one you chose.
- **Export bundles & share links** — contrast recipient-sealed export bundles with bearer share links.
- **Signatures, properties & expiry** — sign and verify through a bridge, inspect properties, and observe destructive expiry.

3.2.3 Mailbox: sending & receiving

- **Mailbox: sending & receiving** — sealing a file to another label, receiving it as that person (the step makes them active with their USB in), and switching back to see the acknowledgement arrive on its own.
- **Reject & acknowledge** — exercise the rejection path and see the sender learn of it as they sign in, with nothing to refresh.
- **Updates & device letters** — see which higher-level workflows use sealed letters and where their controls actually live.

3.2.4 File history

Five short parts on the Activity tab’s **Tracked files** panel. Each one sets up who is active and which drives are in, so they can be taken in any order.

- **1 · Track, sign & fall back** — as Sarah, track a new file, check in an unsigned edit, see sharing fall back to the last trusted revision, sign the edit, and read the result in the activity log’s Revision filter.

- **2 · Hand a file over** — share `budget.txt` with Alice, accept the letter into her own copy as her, then switch back to Sarah, whose history records Alice's signed answer as she signs in.
- **3 · Merges & conflicts** — sign the merge of `forecast.txt` that happened on its own, then review `memo.txt`, whose same-line edits could not merge, and see the reviewer its history names.
- **4 · Gate links & expiry** — as Alice, track a file, link `architecture.md`'s quorum gate to it, open that file and find the unlock in the Security filter, then destroy the tracked file's content and verify the tombstone.
- **5 · Ask for a file or a change** — as Sarah, ask Alice for a change to `budget.txt` with a message, accept the request as Alice, then switch back to Sarah, whose copy records the signed answer as she signs in; the log shows the request and its answer, never the message.

The step card is docked in the top-right corner of the window (bottom edge on a phone), so it never covers the part of the page a step is about; the target is scrolled into view and outlined. **Move** puts the card on the other side and **Hide text** shrinks it to its title.

Finishing a module's last step opens a short completion notice; from there, or from **Exit tutorial** at any point, the **Tutorials & Documentation** button reopens the picker for another module.

4 Where the lab lives

KeyQuorum Lab is published from `lab/` to GitHub Pages and embedded by `bailey-forbes.com`. A GitHub Actions workflow tests, builds, and publishes it from `main` on every change. The lab is a sandboxed machine, not a second implementation of KeyQuorum: it must never reimplement quorum, custody, approval, visibility, bridge, or delivery rules, or add a gate of its own in front of them — if the CLI would allow something on a real machine, the lab allows it too.